

# P&C Insurance Medallion Architecture

## End-to-End Architecture, Agents, Operations, and Deployment Guide

**Audience:** engineers, data engineers, platform administrators, analysts, and new maintainers

**Repository:** [vedavyasgoparaju/pc-insurance-medallion](https://github.com/vedavyasgoparaju/pc-insurance-medallion)

**Current development workspace:** <https://dbc-ec4d2e3d-58c3.cloud.databricks.com>

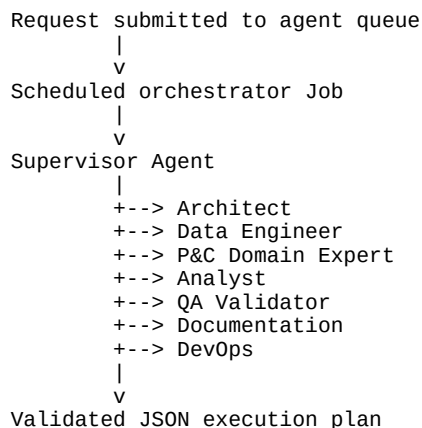
**Document purpose:** explain how the P&C Insurance Medallion platform works from a user request through agent planning, metadata-driven pipeline execution, audit, and Git promotion.

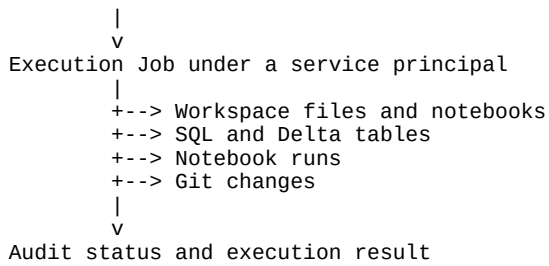
## 1. Executive Summary

This project implements a Property & Casualty insurance data platform on Databricks. It combines:

- A Unity Catalog medallion data model with Bronze, Silver, and Gold layers.
- Metadata-driven Bronze ingestion and Silver transformation control.
- Metadata-driven Gold KPI generation.
- Data quality functions, audit tables, and reconciliation records.
- A Supervisor Agent that coordinates specialist agents.
- An autonomous request queue and orchestrator.
- A controlled execution Job that performs workspace, SQL, notebook, and Git operations.
- Databricks Asset Bundles for repeatable deployment to multiple environments.

The intended operating model is:





The Supervisor reasons and coordinates. The execution Job performs controlled mutations. This separation keeps planning and execution auditable and allows the same code to be promoted across environments.

## 2. Repository Contents

The Git repository contains both application source and deployment source.

### Agent setup and orchestration

| File                       | Responsibility                                                                     |
|----------------------------|------------------------------------------------------------------------------------|
| Supervisor_Agent_Setup.py  | Creates/configures the Supervisor Agent and registers specialist tools.            |
| Architect_Agent.py         | Creates the architecture specialist serving endpoint.                              |
| Data_Engineer_Agent.py     | Creates the data engineering specialist serving endpoint.                          |
| Domain_Expert_Setup.py     | Creates the P&C domain knowledge resource.                                         |
| Analyst_Genie_Setup.py     | Creates the Gold-layer Analyst Genie Space.                                        |
| Orchestrator.py            | Notebook walkthrough and multi-agent demonstration.                                |
| execution/orchestrator.py  | Autonomous queue worker that asks the Supervisor for plans and launches execution. |
| execution/plan_executor.py | Validates and performs approved execution-plan operations.                         |

### Data pipelines

| File                        | Responsibility                                                         |
|-----------------------------|------------------------------------------------------------------------|
| Bronze_Pipeline.py          | Auto Loader ingestion driven by Bronze configuration metadata.         |
| Silver_Pipeline.py          | Silver transformations with metadata-controlled persistence and audit. |
| Silver_Pipeline_Metadata.py | Metadata-driven Silver reference implementation.                       |
| Gold_Pipeline.py            | Gold KPI transformations with metadata-controlled outputs and audit.   |

### Deployment and configuration

| File                                    | Responsibility                                         |
|-----------------------------------------|--------------------------------------------------------|
| databricks.yml                          | Bundle definition, targets, and environment variables. |
| resources/multi_agent_execution_job.yml | Serverless execution and orchestrator Jobs.            |
| sql/01_catalog_schemas.sql              | Catalog and schema bootstrap.                          |

| File                     | Responsibility                             |
|--------------------------|--------------------------------------------|
| sql/02_bronze_tables.sql | Bronze table bootstrap.                    |
| sql/04_gold_tables.sql   | Gold table bootstrap.                      |
| pyproject.toml/uv.lock   | Python and Databricks Connect environment. |
| execution/README.md      | Execution-plan contract and safeguards.    |

The live Databricks Supervisor, serving endpoints, Genie Spaces, App, Unity Catalog objects, and permissions are managed resources. Git stores the code that creates or configures them; it does not contain credentials or a copy of their runtime state.

## 3. Data Architecture

### 3.1 Unity Catalog layout

```
pc_insurance
|-- bronze      Raw ingested source data
|-- silver     Cleansed, conformed dimensions and facts
|-- gold       Business KPIs and executive aggregates
|-- reference  Configuration, audit, reconciliation, and domain reference data
|-- dq         Data quality functions and validation rules
```

### 3.2 Bronze layer

Bronze is metadata-driven through `pc_insurance.reference.bronze_ingestion_config`.

The configuration identifies:

- Logical source name.
- Source directory and file format.
- Source system.
- Bronze target and staging tables.
- Primary key.
- Schema JSON.
- Active/inactive state.
- Load order.

The Bronze pipeline uses Auto Loader, writes staging data, promotes it to target tables, and records:

- `bronze_load_audit`
- `bronze_reconciliation`
- `_load_id`
- `_file_name`
- `source_system`
- `ingestion_timestamp`

### 3.3 Silver layer

Silver is controlled by `pc_insurance.reference.silver_transformation_config`.

The configuration contains:

- Transformation name.
- Source table.
- Target table.
- Staging table.
- Transformation type: DIMENSION\_SCD2, DIMENSION\_TYPE1, FACT, or DEDUP.
- Business key.
- SCD2 tracked columns.
- PII masking rules.
- Join tables.
- Active state.
- Load order.

The pipeline uses the active configuration rows to select target persistence and records seven transformation audit rows in `silver_load_audit`. The Silver model uses SCD2 fields such as `is_current`, `effective_from`, and `effective_to` where appropriate.

### 3.4 Gold layer

Gold is controlled by `pc_insurance.reference.gold_metric_config`.

Each active metric definition contains:

- Metric name.
- Output table.
- Source tables.
- Dimensions.
- Measures.
- Formula or business definition.
- Grain.
- Load order.
- Active state.

Current configured metrics:

| Metric                                | Output                                                      |
|---------------------------------------|-------------------------------------------------------------|
| <code>loss_ratio_by_lob</code>        | Loss, expense, and combined ratios by line of business.     |
| <code>claim_frequency_severity</code> | Claim frequency and severity by line of business and state. |
| <code>retention_by_agent</code>       | Retention and new-business metrics by agent.                |
| <code>premium_growth</code>           | Written and earned premium growth metrics.                  |
| <code>exposure_summary</code>         | Policy and coverage exposure metrics.                       |
| <code>uw_dashboard_summary</code>     | Combined underwriting dashboard metrics.                    |

Each Gold run records source and target counts in `gold_load_audit`. The latest validation produced six successful Gold audit records.

## 4. Agents and Responsibilities

The deployed Supervisor is named **P&C Insurance Medallion Architecture Team**.

### 4.1 Supervisor

The Supervisor is the team lead and router. It:

- 1 Interprets a request.
- 2 Decomposes complex work.
- 3 Routes each part to one or more specialists.
- 4 Requires metadata-driven Silver and Gold designs.
- 5 Synthesizes the specialist responses.
- 6 Returns a strict execution plan to the orchestrator.

The Supervisor should not directly hold broad workspace-admin privileges. Runtime mutations are performed by the execution Job identity.

For mutating requests, the Supervisor's completion policy requires Documentation to update affected documents, QA to validate the change, and DevOps to prepare a final Git commit. The execution plan is not considered complete until those steps are represented and validation succeeds.

### 4.2 Specialist agents

| Agent             | Tool type                      | Responsibilities                                                                                                               |
|-------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Architect         | Serving endpoint               | Medallion design, schemas, data flow, Unity Catalog governance, SCD2, scalability.                                             |
| Data Engineer     | Serving endpoint               | SDP/PySpark/SQL code, metadata-driven pipelines, MERGE logic, Auto Loader, Delta operations.                                   |
| P&C Domain Expert | UC volume / knowledge resource | Policy lifecycle, claims, underwriting, reserving, loss ratios, combined ratios, frequency, severity, retention, NAIC context. |
| Analyst           | Genie Space                    | Gold KPI queries, business metrics, executive summaries, loss and retention analysis.                                          |
| QA Validator      | UC function                    | Data quality validation for premiums, claims, policy IDs, statuses, dates, ratios, and reconciliation.                         |
| Documentation     | Genie Space                    | Architecture documents, data dictionaries, pipeline documentation, and runbooks.                                               |
| DevOps            | Genie Space                    | Git, branch, pull request, DAB, CI/CD, and promotion guidance.                                                                 |
| Workspace actions | Databricks App                 | Controlled workspace writes, SQL, notebook runs, DQ checks, and approved Git operations.                                       |

The workspace-actions App is a controlled capability, not a replacement for the execution Job. Mutating MCP calls may require platform approval depending on the invocation surface. The autonomous queue path avoids relying on direct Supervisor MCP mutation approval.

## 5. Autonomous Execution Flow

### 5.1 Request queue

Requests are stored in:

```
pc_insurance.reference.agent_requests
```

Important columns:

- request\_id
- request\_text
- status
- plan\_json
- execution\_run\_id
- error\_message
- submitted\_at
- updated\_at

Request status values include PENDING, RUNNING, PLANNED, SUCCEEDED, and FAILED.

### 5.2 Orchestrator Job

The orchestrator Job runs on a five-minute schedule. It:

- 1 Reads up to ten pending requests.
- 2 Marks each request RUNNING.
- 3 Sends the request to the Supervisor endpoint.
- 4 Requires JSON with version and operations.
- 5 Validates operation types and workspace paths.
- 6 Supplies the target warehouse ID to SQL operations.
- 7 Submits the plan to the execution Job.
- 8 Waits for the execution result.
- 9 Marks the queue row SUCCEEDED or FAILED.

### 5.3 Execution Job

The execution Job runs the plan under its configured Databricks identity. Supported operations are:

- write\_workspace\_file
- execute\_sql
- run\_notebook

- `git_commit`

Safety controls include:

- Workspace path allowlisting.
- Parent traversal rejection.
- Single-statement SQL enforcement.
- Rejection of DROP, GRANT, and REVOKE statements.
- Explicit operation-type allowlisting.
- Job-level target parameters.

## 5.4 Current Job resources

| Job                                       | Purpose                                                      |
|-------------------------------------------|--------------------------------------------------------------|
| InsuranceModel - Multi-agent orchestrator | Polls requests, consults Supervisor, and launches execution. |
| InsuranceModel - Multi-agent execution    | Executes validated plans on serverless compute.              |

The current dev Job IDs are documented by the Databricks bundle summary and should not be hardcoded into application code. The orchestrator receives the execution Job reference through bundle resource interpolation.

## 6. How to Submit Work

The normal autonomous path is to insert a request into the queue. Example:

```
INSERT INTO pc_insurance.reference.agent_requests
(request_id, request_text, status, submitted_at, updated_at)
VALUES
(
  'silver-claims-metadata-001',
  'Make Silver claims processing metadata driven and add reconciliation checks',
  'PENDING',
  current_timestamp(),
  current_timestamp()
);
```

The scheduled orchestrator will pick it up. Monitor it with:

```
SELECT request_id, status, execution_run_id, error_message,
       submitted_at, updated_at
FROM pc_insurance.reference.agent_requests
ORDER BY submitted_at DESC;
```

For direct development testing only, the orchestrator can also be run with a request parameter:

```
databricks bundle run multi_agent_orchestrator -t dev \
  --params 'request=Run a harmless execution smoke test'
```

The scheduled queue path is preferred for normal autonomous operation.

## 7. Deploying to Another Environment

### 7.1 Prerequisites

For each target environment, create:

- A Databricks CLI profile named staging or prod.
- The pc\_insurance catalog and required schemas.
- SQL warehouse and compute permissions.
- Bronze, Silver, Gold, reference, and DQ tables/functions.
- Agent serving endpoints.
- Domain knowledge resource.
- Analyst, Documentation, and DevOps Genie Spaces.
- Supervisor Agent and its tool registrations.
- Execution service principal permissions.
- Any required Databricks App resources.

## 7.2 Bundle deployment

```
databricks bundle deploy -t staging \
  --var sql_warehouse_id=<staging-warehouse-id> \
  --var supervisor_endpoint=<staging-supervisor-endpoint> \
  --var workspace_root=/Users/<target-user>/InsuranceModel \
    --var allowed_roots=/Users/<target-user>/InsuranceModel,/Repos/<target-user>/pc-insurance-medallion \
    --var repo_path=/Repos/<target-user>/pc-insurance-medallion
```

For production:

```
databricks bundle deploy -t prod \
  --var sql_warehouse_id=<prod-warehouse-id> \
  --var supervisor_endpoint=<prod-supervisor-endpoint> \
  --var workspace_root=/Users/<target-user>/InsuranceModel \
    --var allowed_roots=/Users/<target-user>/InsuranceModel,/Repos/<target-user>/pc-insurance-medallion \
    --var repo_path=/Repos/<target-user>/pc-insurance-medallion
```

Workspace hosts are supplied by the staging and prod Databricks CLI profiles. Credentials are not stored in Git.

## 7.3 Promotion checklist

- [ ] Git commit is reviewed and merged.
- [ ] Target Databricks profile authenticates successfully.
- [ ] Catalog and schema bootstrap is complete.
- [ ] Metadata tables exist and contain active configuration rows.
- [ ] Agent endpoints and Genie resources exist.
- [ ] Supervisor tools point to target resources.
- [ ] Execution service principal has only required target permissions.
- [ ] Bundle validation passes.
- [ ] Bronze smoke test passes.
- [ ] Silver audit and reconciliation rows are successful.
- [ ] Gold audit has one successful row per active metric.
- [ ] Request queue test completes successfully.
- [ ] Rollback commit and Job run history are recorded.

## 8. Permissions and Security

The execution identity, not the conversational Supervisor, should own mutation permissions.

Required permissions are scoped to the target environment:

- Workspace CAN\_EDIT on approved project paths.
- SQL warehouse CAN\_USE.
- Unity Catalog USE CATALOG and USE SCHEMA.
- SELECT, MODIFY, and CREATE TABLE where required.
- EXECUTE on DQ functions.
- Job permissions to submit and monitor runs.
- Git/Repos permissions for the approved repository.

Do not grant workspace-admin or account-admin permissions to the agent runtime. Keep the path allowlist, SQL guardrails, queue audit, Job audit, and Git history enabled.

## 9. Operations and Troubleshooting

### Request remains PENDING

- Check the orchestrator Job schedule and latest runs.
- Confirm the Job is not paused.
- Confirm the orchestrator service identity can query agent\_requests.
- Confirm the target SQL warehouse is available.

### Request becomes FAILED

Inspect:

```
SELECT request_id, status, error_message, plan_json
FROM pc_insurance.reference.agent_requests
WHERE status = 'FAILED'
ORDER BY updated_at DESC;
```

Also inspect the execution Job run referenced by execution\_run\_id.

### Silver failure

Check:

```
SELECT transformation_name, status, error_message,
       source_row_count, staging_row_count, target_row_count_after
FROM pc_insurance.reference.silver_load_audit
ORDER BY load_start_time DESC;
```

Then check configuration:

```
SELECT *
FROM pc_insurance.reference.silver_transformation_config
WHERE is_active = true
ORDER BY load_order;
```

## Gold failure

Check:

```
SELECT metric_name, output_table, status, error_message,
       source_row_count, target_row_count
FROM pc_insurance.reference.gold_load_audit
ORDER BY load_start_time DESC;
```

Then validate active metric metadata:

```
SELECT metric_name, output_table, source_tables, dimensions,
       measures, formula, grain, load_order
FROM pc_insurance.reference.gold_metric_config
WHERE is_active = true
ORDER BY load_order;
```

## Deployment failure

Run:

```
databricks bundle validate -t <target>
databricks bundle summary -t <target>
```

Confirm the target profile, warehouse ID, Supervisor endpoint, workspace root, and allowed roots were supplied correctly.

# 10. Ownership Model

| Concern                            | Owner                             |
|------------------------------------|-----------------------------------|
| Request interpretation and routing | Supervisor Agent                  |
| Architecture decisions             | Architect Agent                   |
| Pipeline implementation            | Data Engineer Agent               |
| Insurance definitions and formulas | P&C Domain Expert                 |
| KPI analysis                       | Analyst Agent                     |
| Data quality and reconciliation    | QA Validator                      |
| Technical documentation            | Documentation Agent               |
| Git and promotion workflow         | DevOps Agent                      |
| Plan validation and Job trigger    | Autonomous orchestrator           |
| Workspace and data mutations       | Execution Job service principal   |
| Environment credentials and ACLs   | Databricks platform administrator |
| Git history and review             | Engineering team                  |

# 11. New Maintainer Quick Start

- 1 Clone the repository.
- 2 Read this guide and README.md.
- 3 Configure the Databricks CLI profile.
- 4 Run `databricks bundle validate -t dev`.
- 5 Inspect the Supervisor tools and target Job summary.

- 6 Submit a harmless request to the queue.
- 7 Follow the request status and execution run.
- 8 Verify Silver and Gold audit records.
- 9 Review Git changes before promotion.
- 10 Deploy staging only after the target resources and permissions exist.

The platform is considered healthy when a queued request moves from PENDING to SUCCEEDED, the execution Job completes successfully, and the corresponding data-layer audit records are present.